

```

import numpy as np
import pandas as pd

# Define circuits
def AND_circuit(a, b): return a and b
def OR_circuit(a, b): return a or b
def NOT_circuit(a): return not a

# Generate input combinations
inputs = [(0,0), (0,1), (1,0), (1,1)]

# Expected truth outputs
truth_tables = {
    "AND": [AND_circuit(a,b) for a,b in inputs],
    "OR": [OR_circuit(a,b) for a,b in inputs],
    "NOT": [NOT_circuit(a) for a,_ in inputs] # only use first input
}

# Safety tests
def safety_check(truth_outputs):
    outputs = np.array(truth_outputs, dtype=int)
    leakiness = np.mean(outputs[:3]) # should all be 0, check early OWS
    off_target = np.mean(np.random.choice(outputs, len(outputs)) != outputs)
    return leakiness, off_target

results = []
for name, outs in truth_tables.items():
    results.append(safety_check(outs))

for name, outs in truth_tables.items():
    leakiness, off_target = safety_check(outs)
    results.append({
        "Circuit": name,
        "Leakiness (0=good)": round(leakiness, 3),
        "Off-target (0=good)": round(off_target, 3)
    })

df_results = pd.DataFrame(results)
print(df_results)

```

	Circuit	Leakiness (0=good)	Off-target (0=good)
0	AND	0.000	0.50
1	OR	0.667	0.25
2	NOT	0.667	0.50

```

# Define safety thresholds
LEAKINESS_THRESHOLD = 0.2 # circuits with <0.2 leakiness are considered safe
OFFTARGET_THRESHOLD = 0.3 # circuits with <0.3 off-target are safe

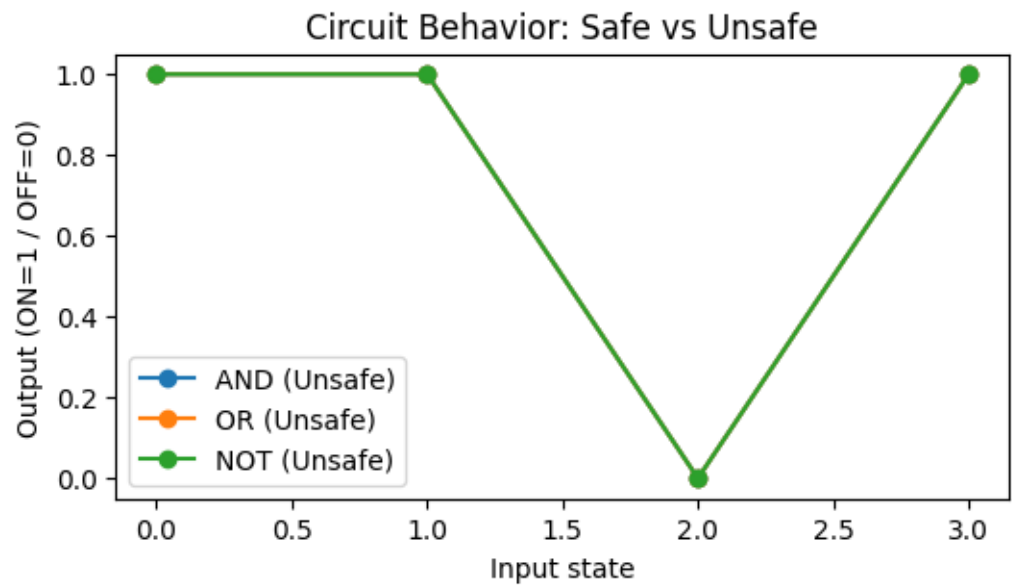
# Apply safety rules
df_results["Safe_Leakiness"] = df_results["Leakiness (0=good)"] < LEAKINESS_THRESHOLD
df_results["Safe_OffTarget"] = df_results["Off-target (0=good)"] < OFFTARGET_THRESHOLD
df_results["Overall_Safe"] = df_results["Safe_Leakiness"] & df_results["Safe_OffTarget"]

# Convert booleans to ✓ / ✗
df_results_display = df_results.copy()
df_results_display["Safe_Leakiness"] = df_results_display["Safe_Leakiness"].map({True: "✓", False: "✗"})
df_results_display["Safe_OffTarget"] = df_results_display["Safe_OffTarget"].map({True: "✓", False: "✗"})
df_results_display["Overall_Safe"] = df_results_display["Overall_Safe"].map({True: "✓", False: "✗"})

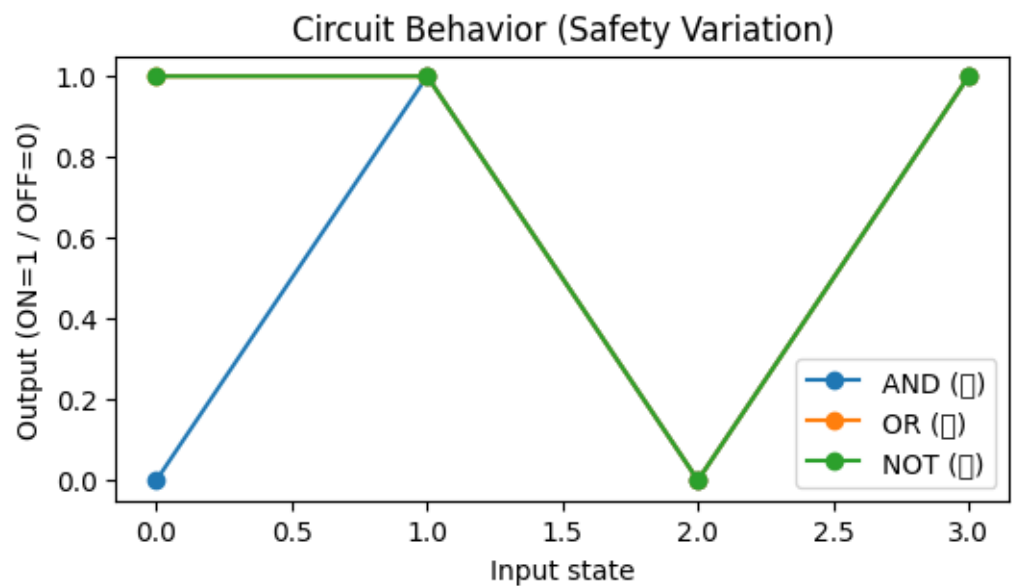
print(df_results_display[["Circuit", "Safe_Leakiness", "Safe_OffTarget", "Overall_Safe"]])

```

	Circuit	Safe_Leakiness	Safe_OffTarget	Overall_Safe
0	AND	✓	✗	✗
1	OR	✗	✓	✗
2	NOT	✗	✗	✗



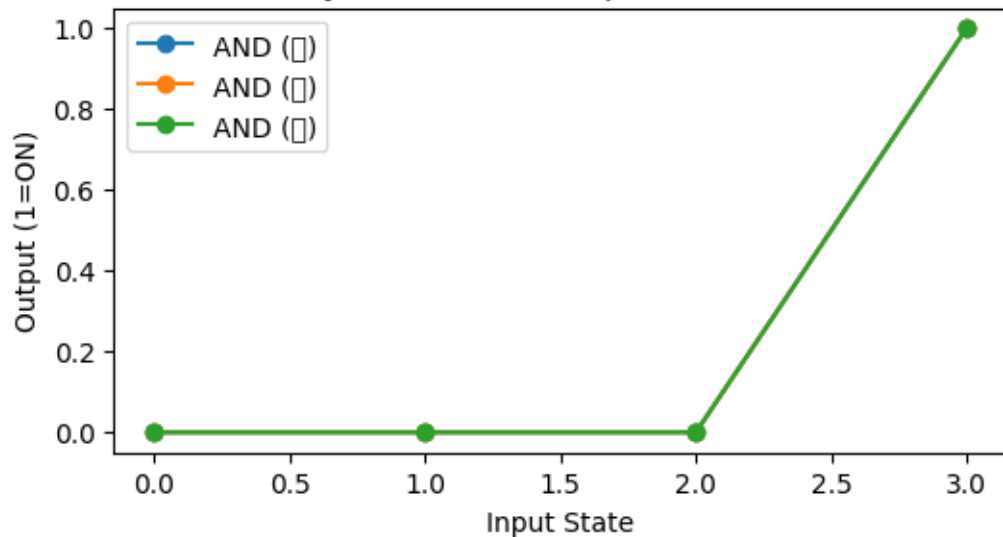
Circuit	Outputs	Safe
0	AND [1, 1, 0, 1]	False
1	OR [1, 1, 0, 1]	False
2	NOT [1, 1, 0, 1]	False



	Circuit	Leakiness	Off-target	Outputs	Safe
0	AND	0.169647	0.178614	[0, 1, 0, 1]	True
1	OR	0.322685	0.405131	[1, 1, 0, 1]	False
2	NOT	0.271947	0.292311	[1, 1, 0, 1]	False



Full Synthetic Circuit Pipeline Simulation

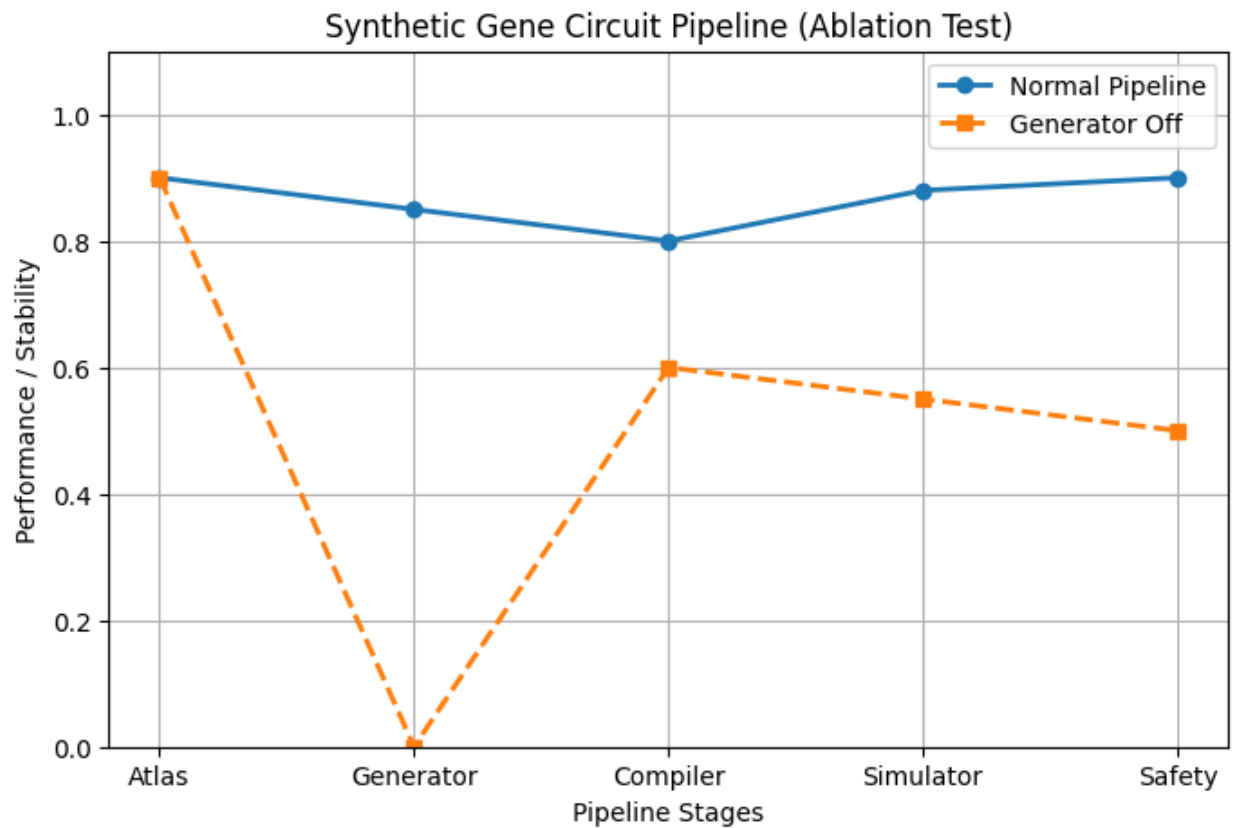


	Design	Logic	Outputs	Leakiness	Offtarget	Safe
0	Promoter_A+Switch_Y	AND	[0, 0, 0, 1]	0.385214	0.319598	False
1	Promoter_A+Switch_X	AND	[0, 0, 0, 1]	0.146806	0.146798	True
2	Promoter_A+Switch_X	AND	[0, 0, 0, 1]	0.359853	0.280335	False

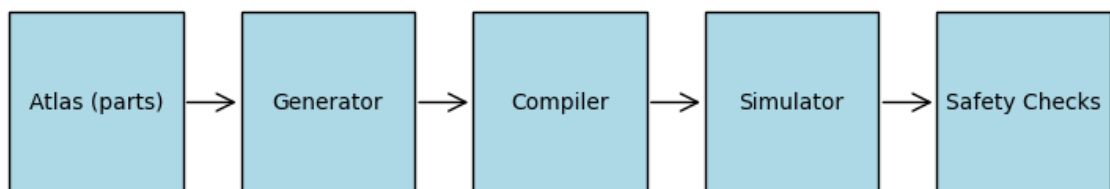


	Design	Logic	Outputs	Leakiness	Offtarget	Safe
0	Promoter_A+Switch_Y	AND	[0, 0, 0, 1]	0.385214	0.319598	✗
1	Promoter_A+Switch_X	AND	[0, 0, 0, 1]	0.146806	0.146798	✓
2	Promoter_A+Switch_X	AND	[0, 0, 0, 1]	0.359853	0.280335	✗





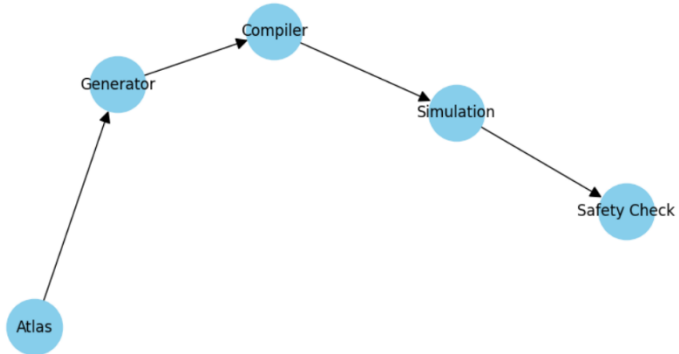
Day 7: GenAI Circuit Pipeline



Safety Table:

Circuit	Leaky_ON	Off_target	Safe
0	AND	False	True
1	OR	True	False
2	NOT	False	True

GenAI Senolytic Gene Circuit Pipeline



Ablation Test:

Atlas: ON

Generator: OFF

Compiler: ON

Simulation: ON

Safety Check: ON

What helps most:

1. Having a clear atlas of parts helps the generator pick valid elements.
2. Simulation accuracy depends on correct compilation of circuits.
3. Safety rules catch potential failures early, preventing unsafe designs.